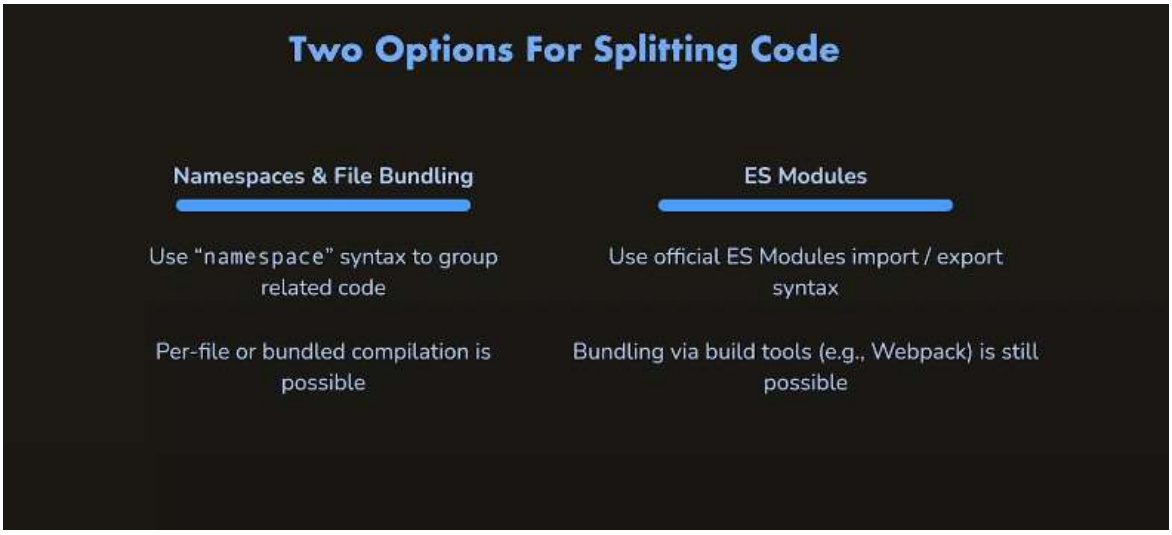
Section 14 - Modules and Namespaces

2026年6月10日　12:38



Splitting TypeScript Code Across Multiple Files

When working with larger TypeScript projects, you'll need to split your code across multiple files and then connect them together. TypeScript gives you **two main approaches** for this:

1. **Namespaces** (TypeScript-specific)
2. **ES Modules** (Standard JavaScript approach)

1. Namespaces + File Bundling (TypeScript Approach)

TypeScript provides a feature called **namespaces** to group related code together, even when that code lives in different files.

- You use the namespace keyword to logically group code.
- You can reference code from other files using the /// <reference /> directive or by organizing files properly.
- With the help of the tsconfig.json file, TypeScript can **bundle** all these separate files into a **single JavaScript output file** during compilation.

This approach was especially useful before modern JavaScript module systems became widespread. It allows you to split code during development while ending up with one consolidated file for production.

2. ES Modules (Modern Standard Approach)

TypeScript also fully supports the official **ES Modules** syntax (import / export), which is the current standard way of splitting and sharing code in JavaScript.

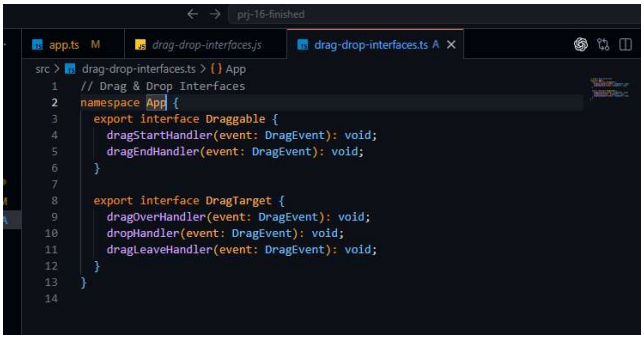
- This is **not** TypeScript-specific — it's the native JavaScript module system.
- You use export to make features available from a file and import to use them in another file.
- Modern bundlers (like Webpack, Vite, esbuild, etc.) are typically used to bundle these modules for production.

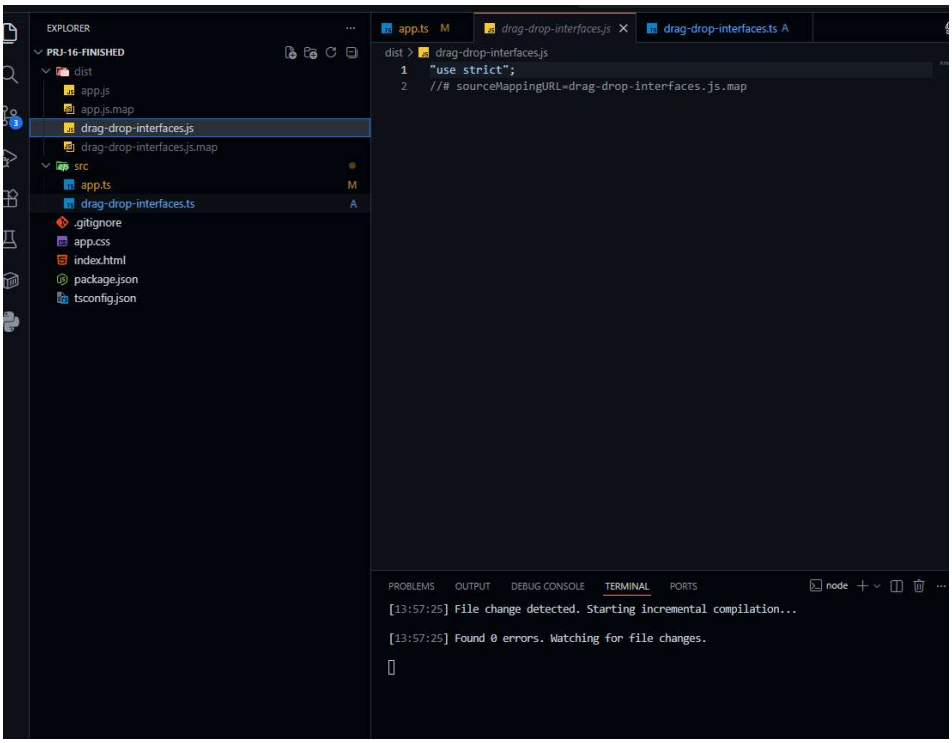
Summary

Approach	TypeScript Specific	Modern Standard	Output Strategy	Best For
Namespaces	Yes	No	Built-in bundling (tsconfig)	Legacy projects, simple apps
ES Modules	No (but supported)	Yes	Usually requires a bundler	Modern web/apps, scalability

Key Takeaway: You can split your TypeScript code across many files for better organization and maintainability. Then, depending on your needs and project age, you can either use **namespaces** (with TypeScript's built-in bundler) or the modern **ES Modules** syntax (with tools like Vite/Webpack).

Working with Namespace

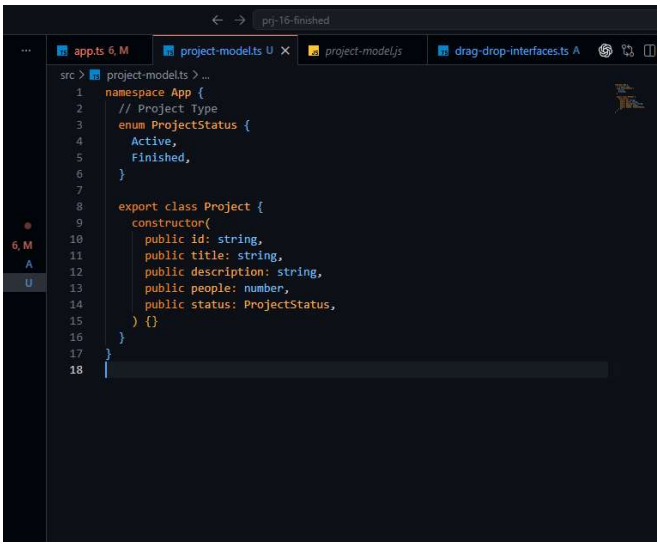




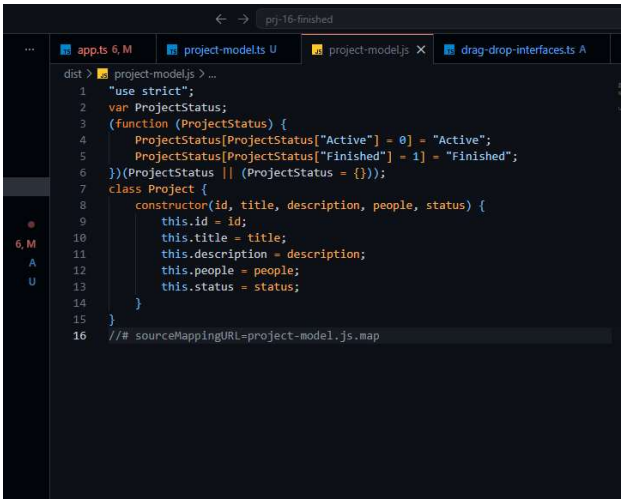
There is no js equivalent for interfaces. That's why this compiled file is empty.

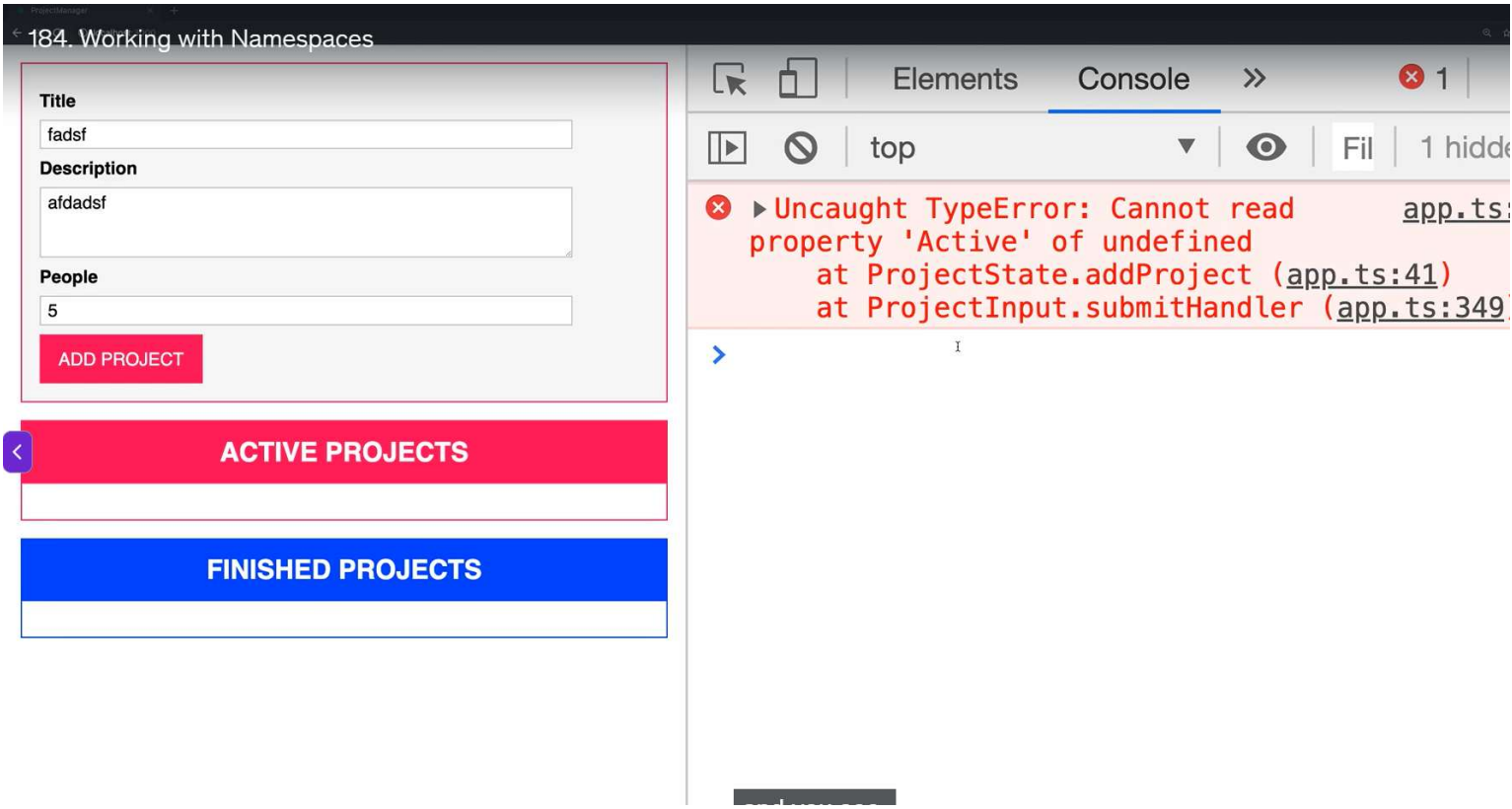
But what if we create another namespace in a separate file and that contains ts code which has js equivalent ?

For example, in the project-model.ts file, we have defined enum and class,

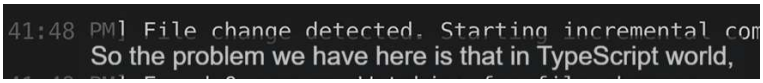


Now, in the compiled code, we can see it's corresponding js code. But the question is will it work here?

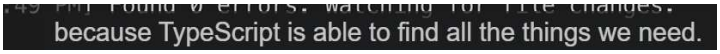


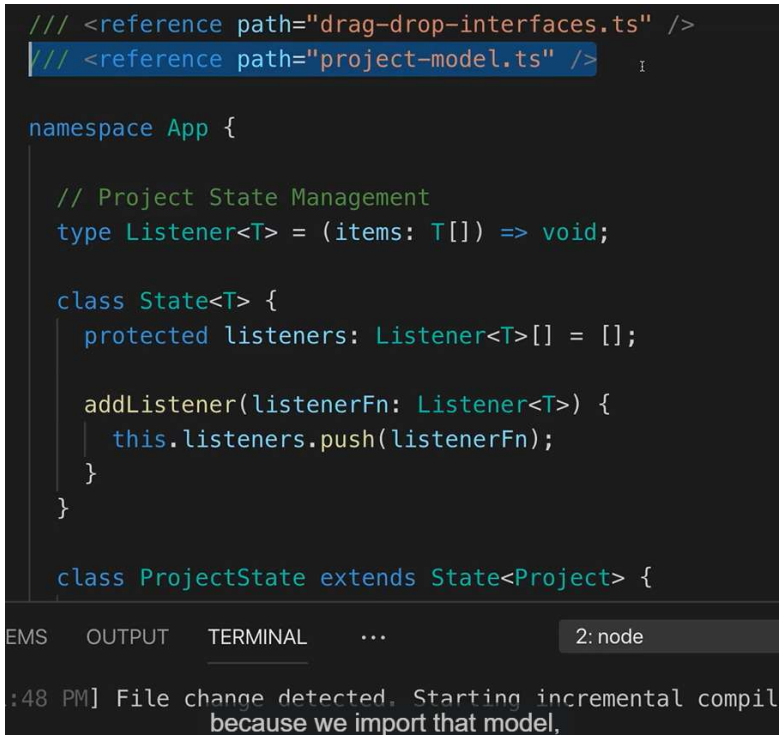


We can see an error, even though TypeScript is not complaining.



Everything is great





but import here really just means

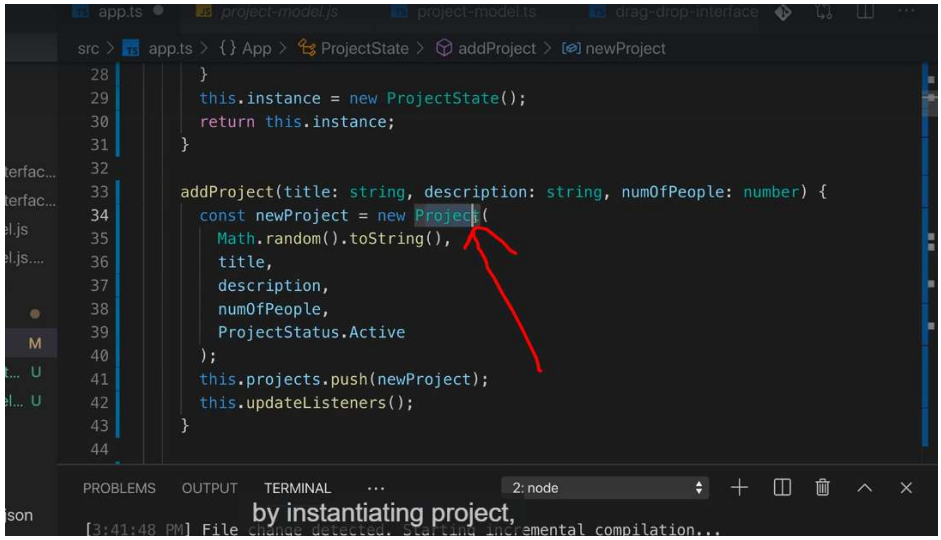
we tell TypeScript where to find that type

once it is compiled to JavaScript,

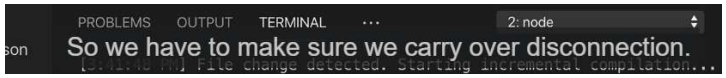
This connection is basically destroyed.

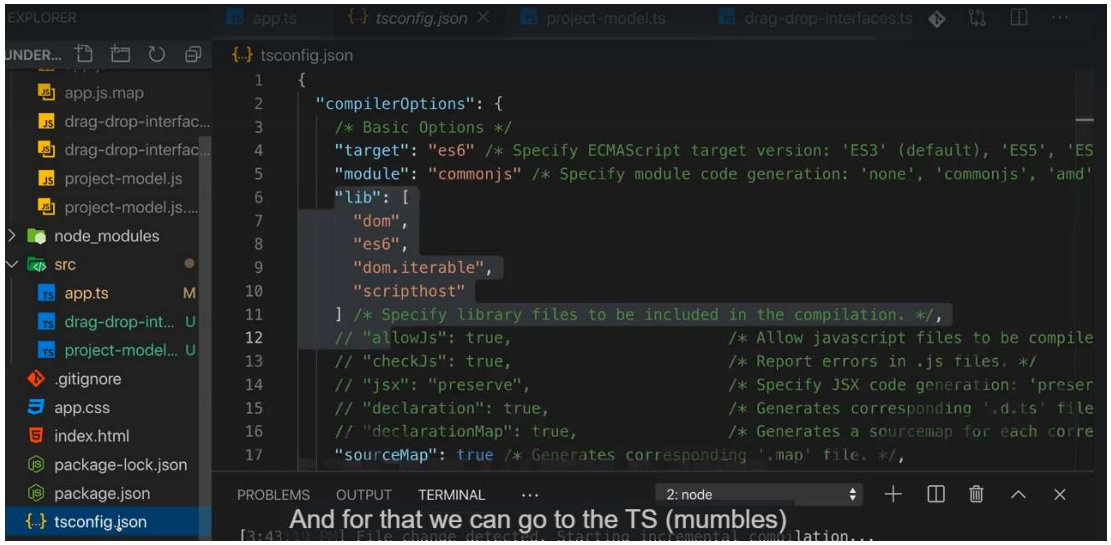
So in JavaScript code when is executes

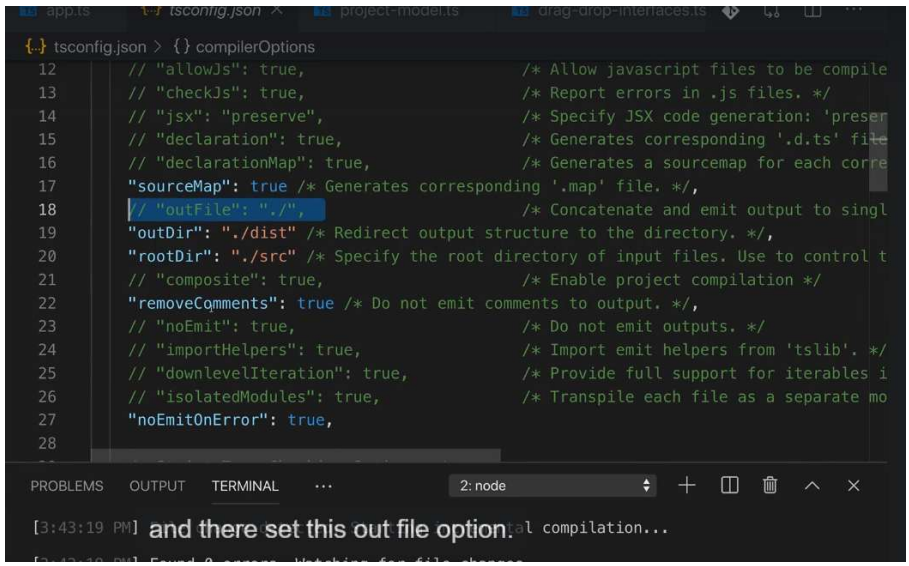
cp /tmp/diabetes-slides.html /mnt/c/Users/BJIT/Documents/diabetes-slides.html && echo "Copied OK" && wc -l /tmp/diabetes-slides.html

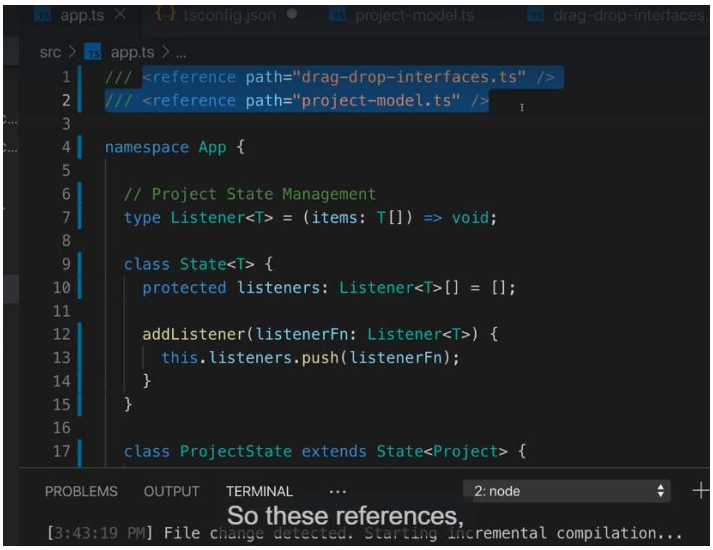


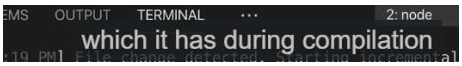
or constructor function.

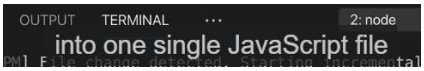


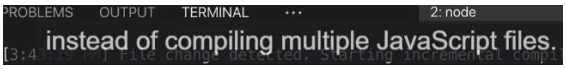


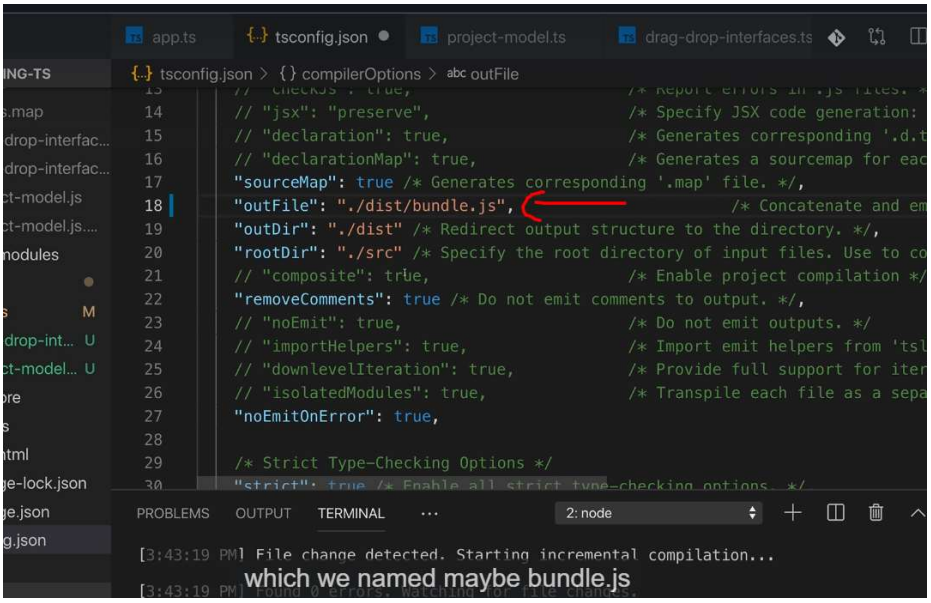


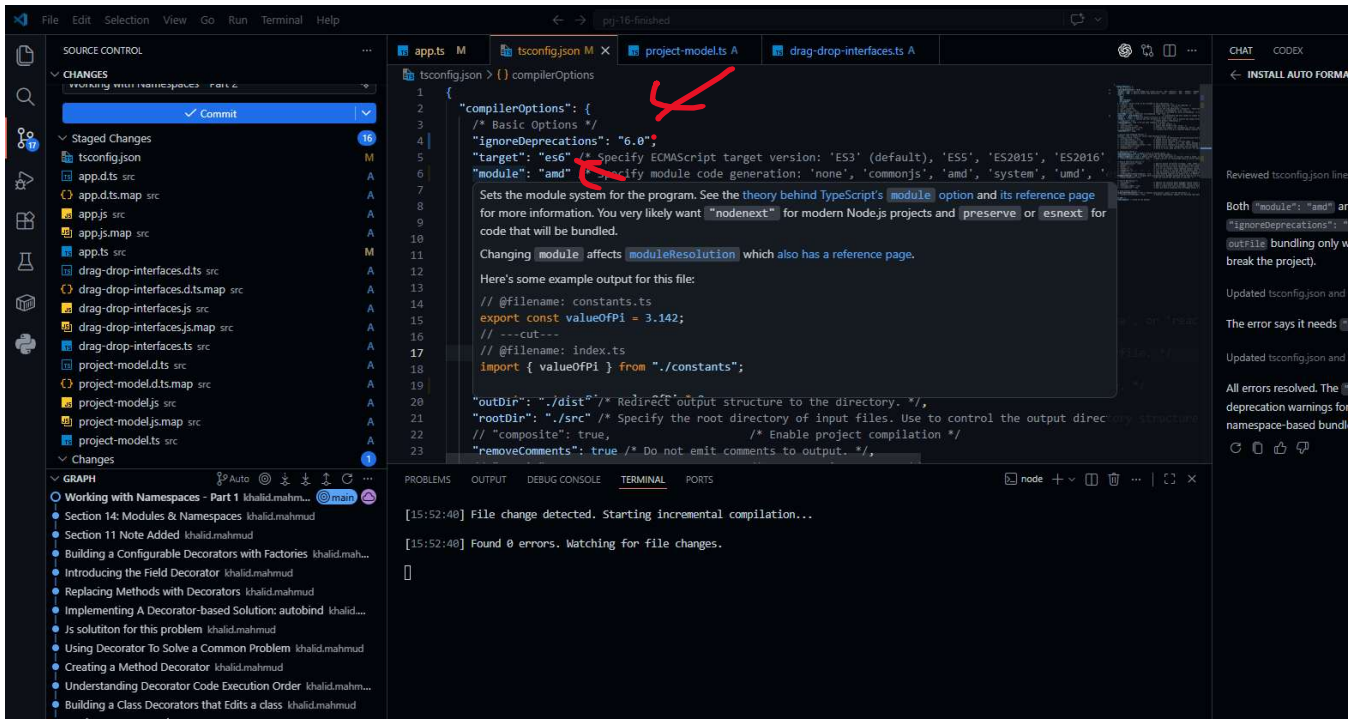

app.ts
tsconfig.json
project-model.ts
drag-drop-interfaces
src > app.ts > ...
/// <reference path="drag-drop-interfaces.ts" />
/// <reference path="project-model.ts" />
namespace App {
// Project State Management
type Listener<T> = (items: T[]) => void;
class State<T> {
protected listeners: Listener<T>[] = [];
addListener(listenerFn: Listener<T>) {
this.listeners.push(listenerFn);
}
}
class ProjectState extends State<Project> {
PROBLEMS OUTPUT TERMINAL
2: node
So these references,
[3:43:19 PM] File change detected. Starting incremental compilation...

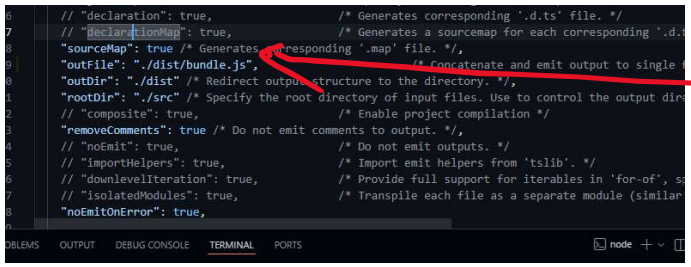

OUTPUT TERMINAL
2: node
which it has during compilation


OUTPUT TERMINAL
2: node
into one single JavaScript file

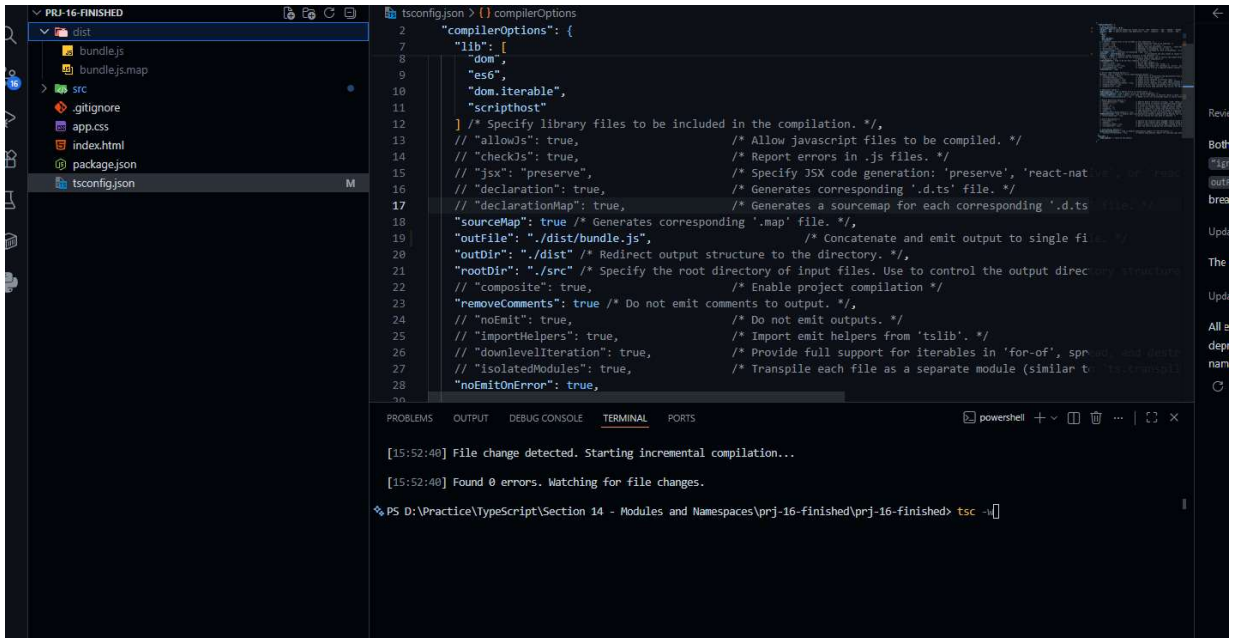

PROBLEMS OUTPUT TERMINAL
2: node
instead of compiling multiple JavaScript files.

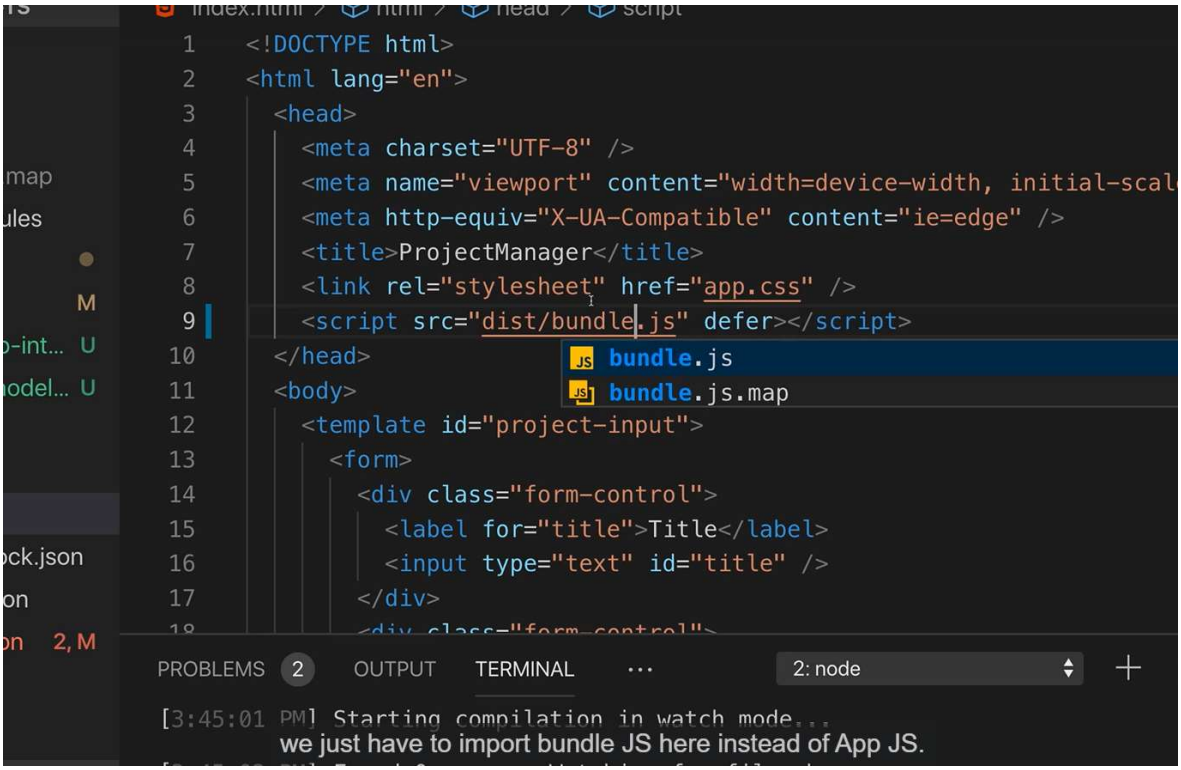

app.ts
tsconfig.json
project-model.ts
drag-drop-interfaces.ts
tsconfig.json > { } compilerOptions > abc outFile
// "jsx": "preserve", /* Specify JSX code generation:
// "declaration": true, /* Generates corresponding '.d.t
// "declarationMap": true, /* Generates a sourcemap for eac
"sourceMap": true /* Generates corresponding '.map' file. */,
"outFile": "./dist/bundle.js", /* Concatenate and em
"outDir": "./dist" /* Redirect output structure to the directory. */,
"rootDir": "./src" /* Specify the root directory of input files. Use to co
// "composite": true, /* Enable project compilation */
"removeComments": true /* Do not emit comments to output. */,
// "noEmit": true, /* Do not emit outputs. */
// "importHelpers": true, /* Import emit helpers from 'tsl
// "downlevelIteration": true, /* Provide full support for iter
// "isolatedModules": true, /* Transpile each file as a sepa
"noEmitOnError": true,
/* Strict Type-Checking Options */
PROBLEMS OUTPUT TERMINAL
2: node
[3:43:19 PM] File change detected. Starting incremental compilation...
which we named maybe bundle.js





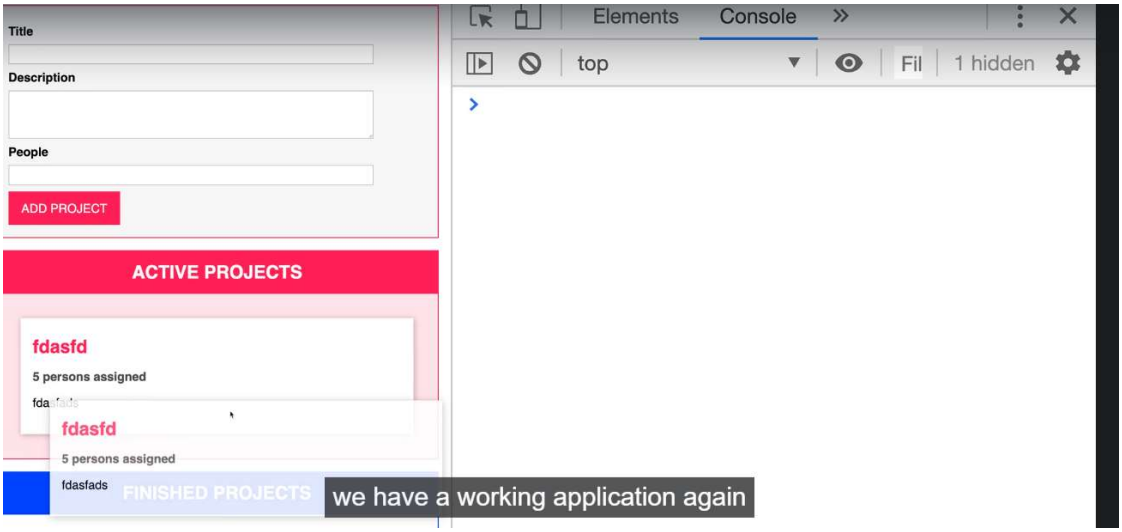
Now if we re-start our server, only bundle.js file is created




index.html > html > head > script
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scal
<meta http-equiv="X-UA-Compatible" content="ie=edge" />
<title>ProjectManager</title>
<link rel="stylesheet" href="app.css" />
<script src="dist/bundle.js" defer></script>
</head>
<body>
<template id="project-input">
<form>

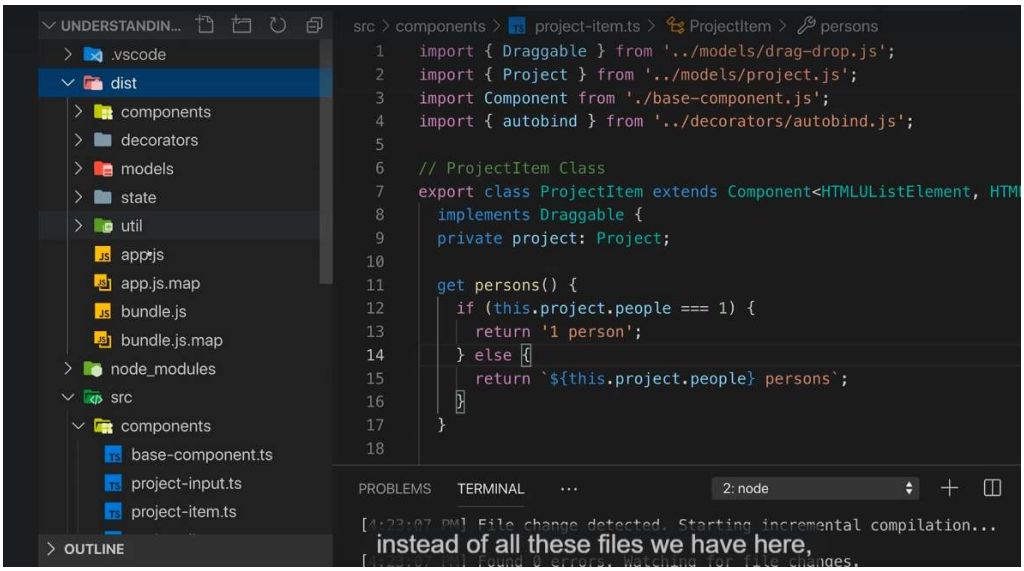
<label for="title">Title</label>
<input type="text" id="title" />

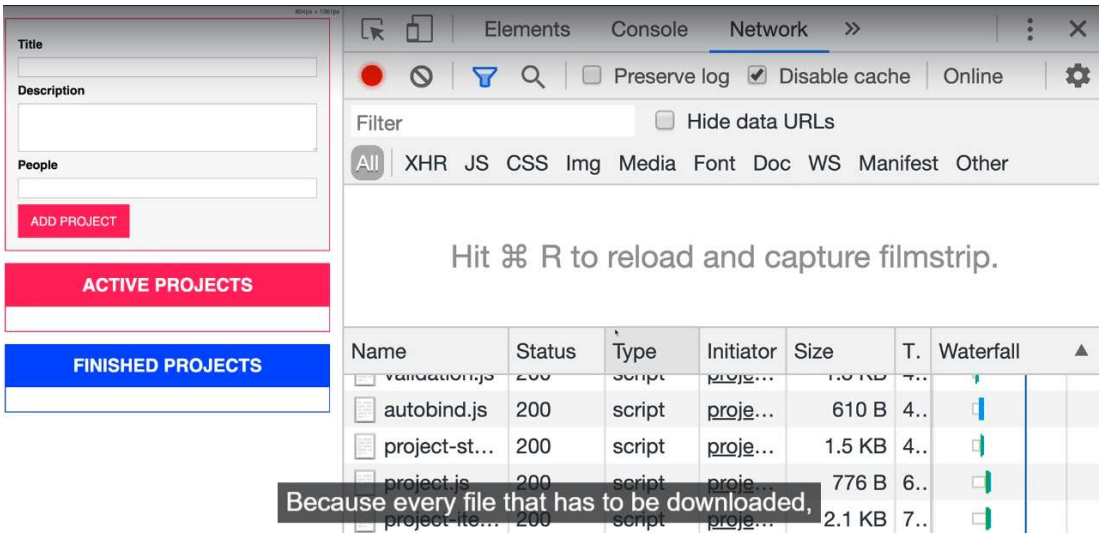
bundle.js
bundle.js.map
PROBLEMS 2
OUTPUT
TERMINAL
2: node
[3:45:01 PM] Starting compilation in watch mode...
we just have to import bundle JS here instead of App JS.


Title
Description
People
ADD PROJECT
ACTIVE PROJECTS
fdasfd
5 persons assigned
fdasfd
5 persons assigned
fdasfads
FINISHED PROJECTS
Elements
Console
top
Fil
1 hidden
we have a working application again

Wrap Up

```
src > components > project-item.ts > ...
1   import { Draggable } from '../models/drag-drop.js';
2   import { Project } from '../models/project.js';
3   import Component from './base-component.js';
4   import { autobind } from '../decorators/autobind.js';
5
6   // ProjectItem Class
7   export class ProjectItem extends Component<HTMLULListElement, HTM
8     implements Draggable {
9     private project: Project;
10
11    get persons() {
12      if (this.project.people === 1) {
13        return '1 person';
14      } else {
15        return `${this.project.people} persons`;
16      }
17    }
```





Why Modules Matter in TypeScript

And that's it for modules!

Using modules is an extremely useful and important feature in TypeScript. It allows you to write **more maintainable and manageable code**. Putting everything in a single file quickly becomes a mess and is only realistic for very tiny projects.

Whether you choose **namespaces** or **ES modules**, both approaches let you:

- Keep individual files relatively small and focused.
- Still build one large, cohesive application in the end.

Recommendation: Use ES Modules

When comparing the two approaches, the **clear recommendation** is to use **ES Modules**.

Why ES Modules are better:

- You get stronger type safety.
- Every file must **explicitly** declare what it imports and exports.
- Dependencies are clear and visible.

Problems with Namespaces:

- One file can import something that another file silently depends on.
- If you remove the file that does the importing, other files can break without any compile-time warning.
- These issues often only surface at runtime.

When to still use Namespaces?

- In very small projects.
- In legacy setups where you cannot use ES modules or a bundler.

Browser Compatibility & Bundling

The ES Modules syntax we're using right now only works in **modern browsers**. Older browsers like Internet Explorer 9 do not support it (and may not support other modern JavaScript features either).

Solution: Use a bundler

To make your TypeScript application work across all browsers, you should use a bundling tool such as **Webpack** (or modern alternatives like Vite, esbuild, etc.).

Benefits of bundling:

1. **Browser Compatibility** — The bundler converts modern module syntax into code that older browsers can understand.
2. **Performance** — Instead of downloading many small files, the browser only needs to fetch **one** JavaScript file.
    - Every file request has overhead (connection setup, HTTP request latency).
    - You can see this clearly in the browser's **Network tab** as a waterfall chart — each request has extra "white space" (setup time) at the beginning.

Even if browser support wasn't an issue, reducing the number of requests is generally better for performance.

Summary

- **Development**: Split your code into many small, well-organized TypeScript files using ES Modules.
- **Production**: Bundle everything into one (or very few) JavaScript files using Webpack or similar tools.

This gives you the best of both worlds: excellent developer experience with strong typing + optimal performance and compatibility for users.

Would you like me to format the next section, or combine everything we have so far into one complete, well-structured guide